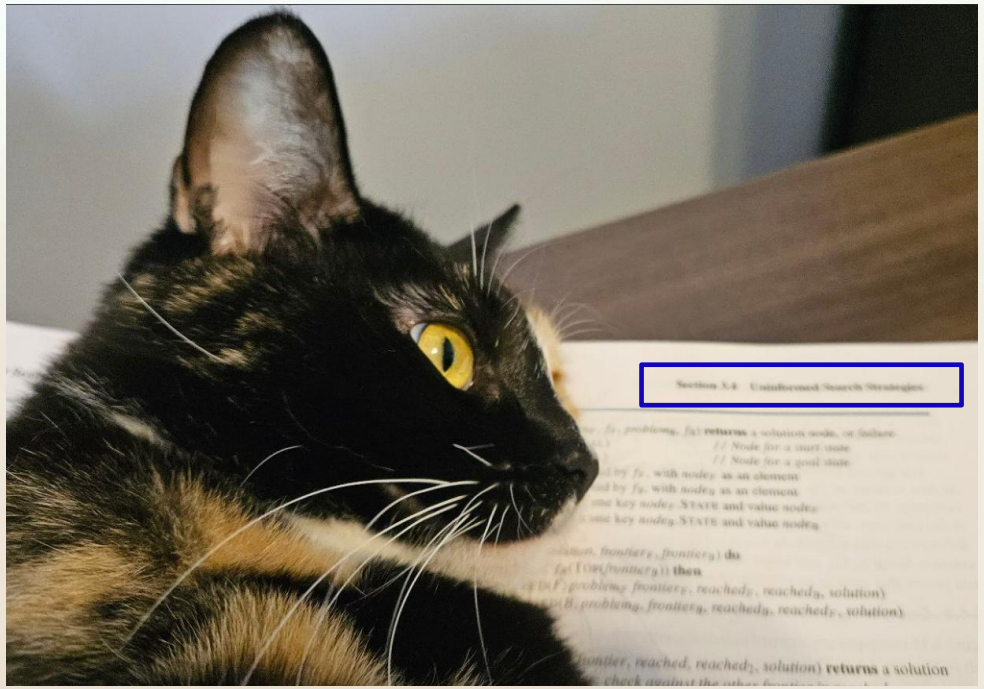
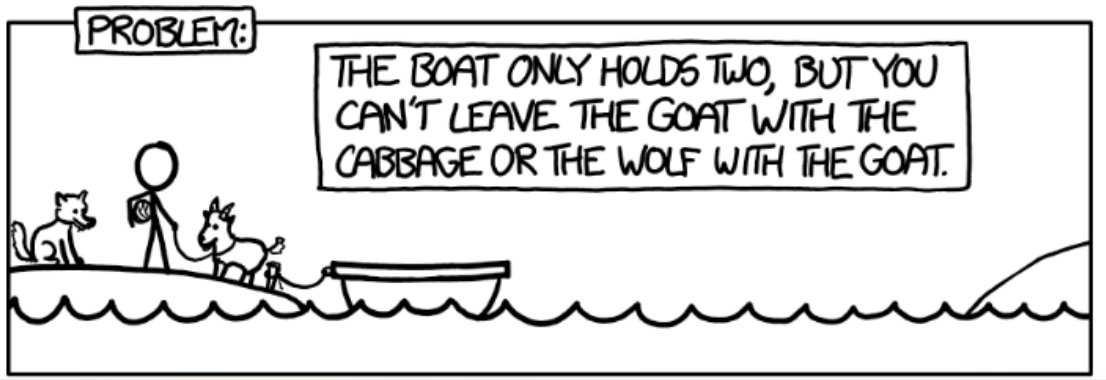
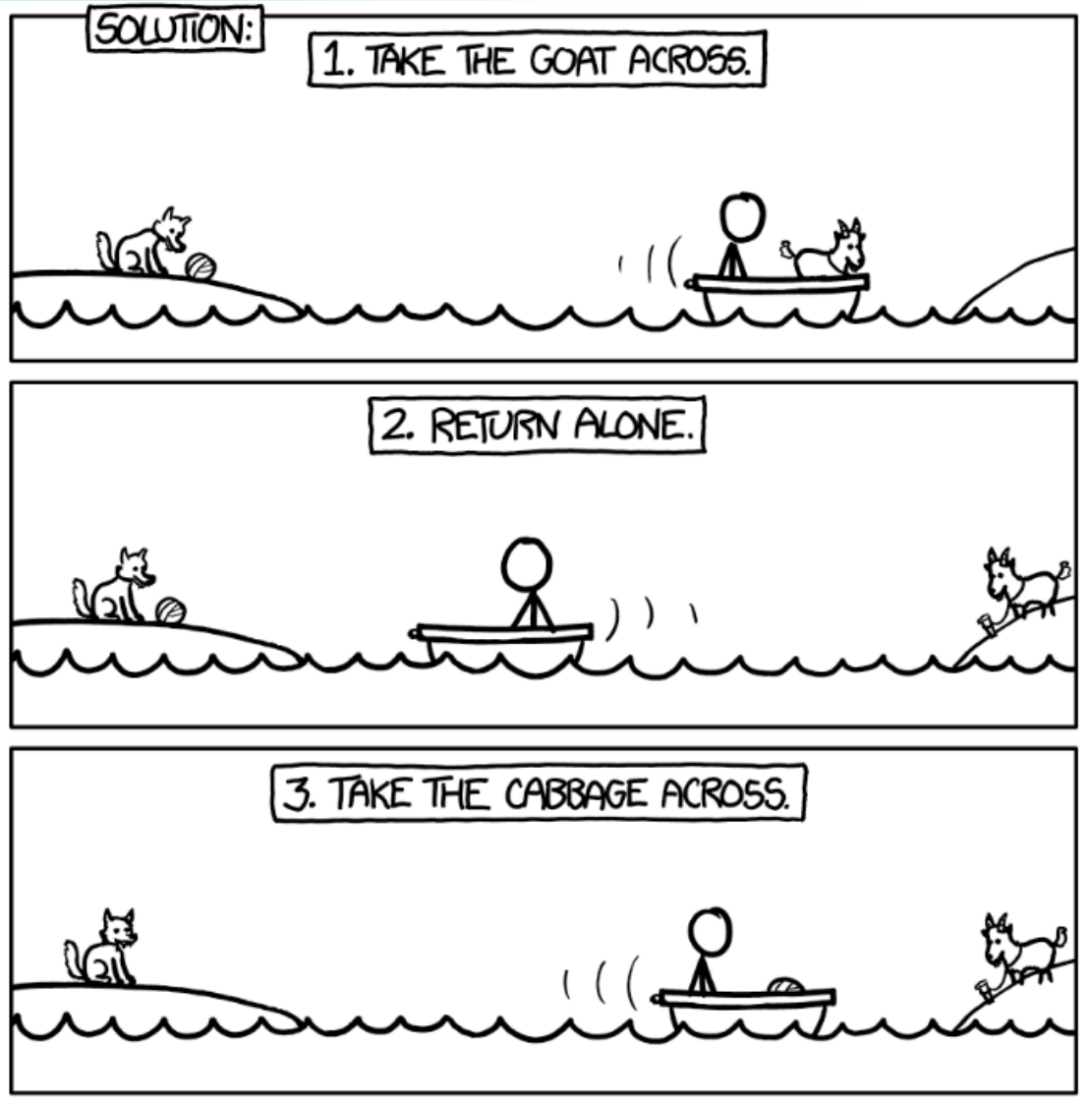
Uniformed Search



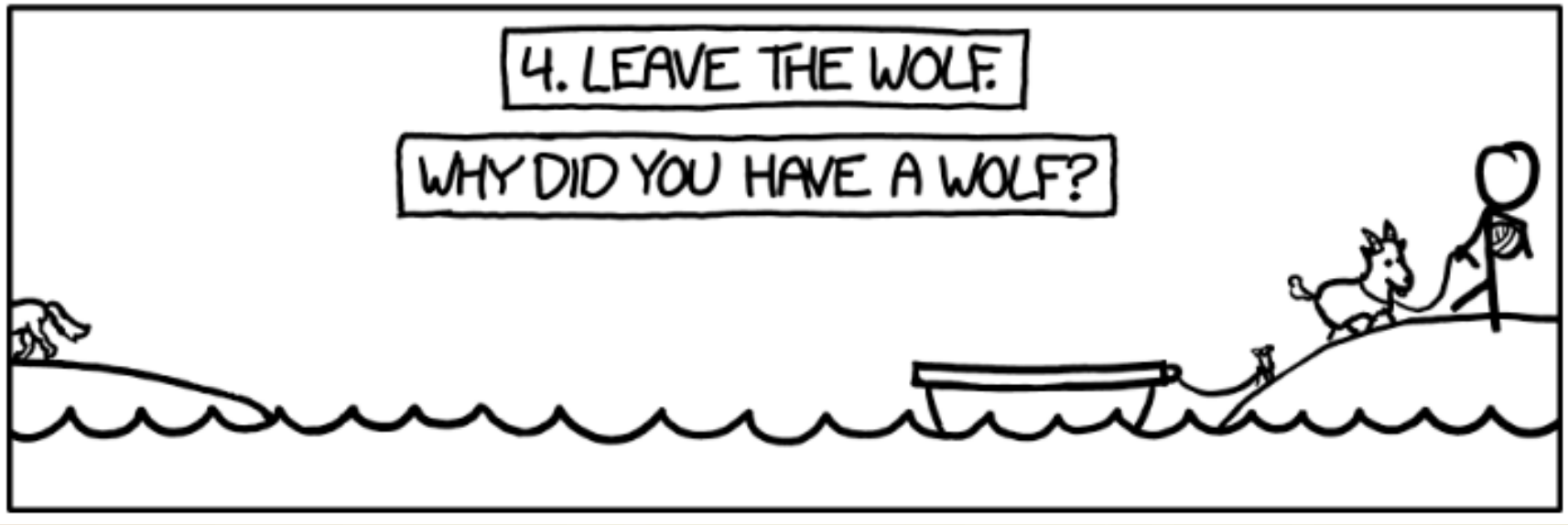
Poppy is into it.

A Motivating Example

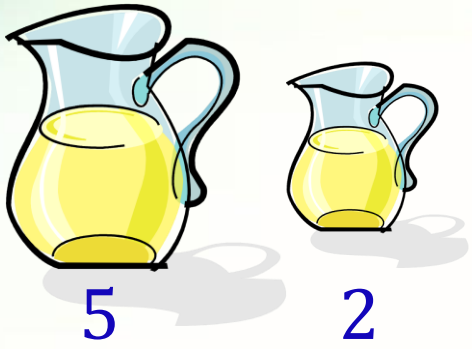




https://xkcd.com/1134/


4. LEAVE THE WOLF.
WHY DID YOU HAVE A WOLF?

Search Example: Jugs



Given: 5-gallon jug and 2-gallon jug

State Representation: $(f, t) =$ (#gallons in 5-gal, #gallons in 3-gal)

Goal: Acquire 1 gallon **Goal States:** $(*, 1)$ or $(1, *)$, where $*$ refers to any amount in the jug.

Initial State: $(5,0)$

Transitions:

Empty the 5-gallon jug	$(f, t) \rightarrow (0, t)$
Empty the 2-gallon jug	$(f, t) \rightarrow (f, 0)$
Pour 2-gallon jug into 5-gallon jug	$(f, 2) \rightarrow (f + 2, 0)$, where $f \leq 3$
Pour 5-gallon jug into 2-gallon jug	$(f, 0) \rightarrow (f - 2, 2)$, where $f \geq 2$
Pour 1 gallon from 5-gallon jug into 2-gallon jug	$(1,0) \rightarrow (0,1)$

(These transitions imply you cannot measure or pour a specific amount of water from a jug.)

BFS Search of Jugs

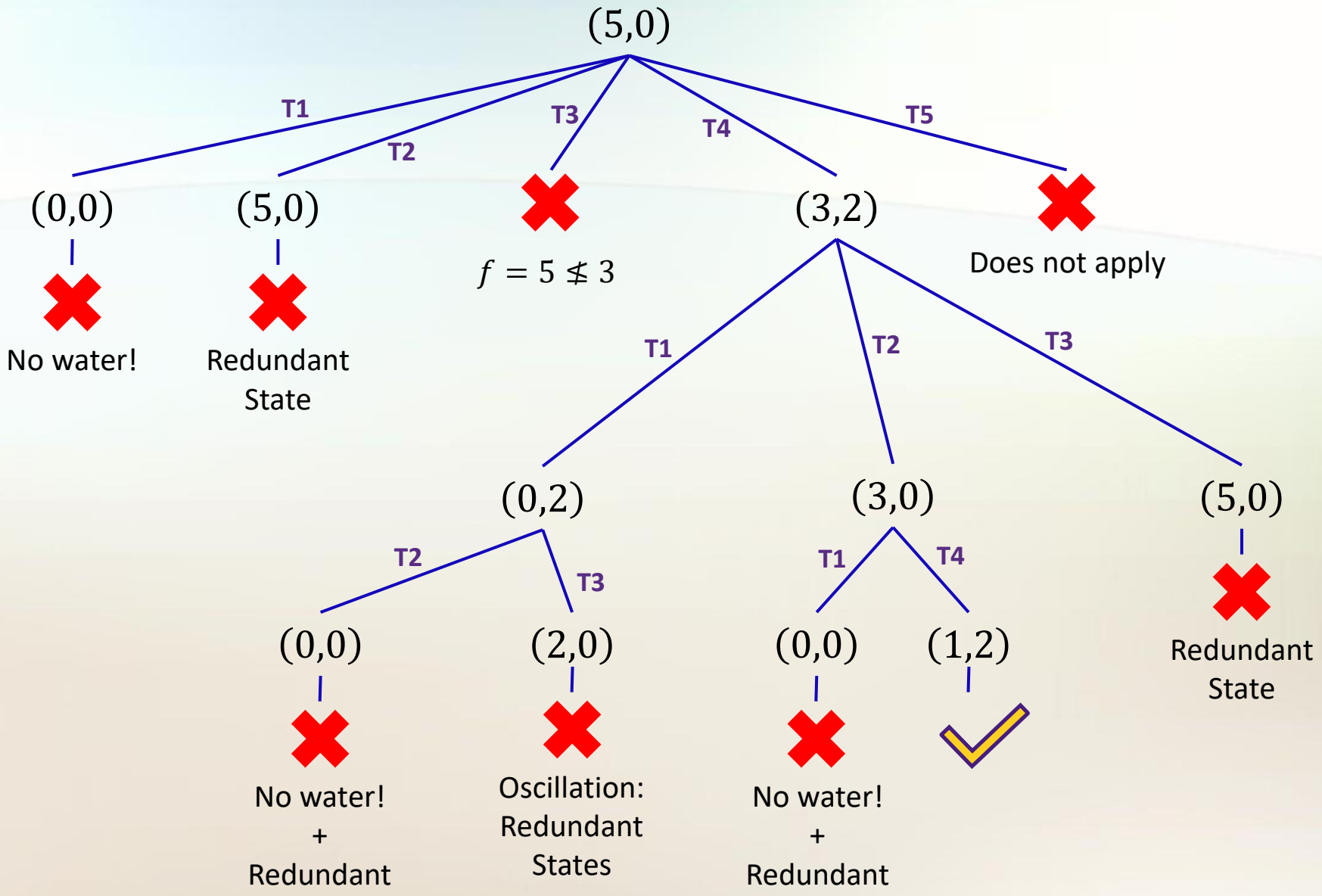
T1 $(f,t) \rightarrow (0,t)$

T2 $(f,t) \rightarrow (f,0)$

T3 $(f,2) \rightarrow (f+2,0)$, where $f \leq 3$

T4 $(f,0) \rightarrow (f-2,2)$, where $f \geq 2$

T5 $(1,0) \rightarrow (0,1)$



Search is Almost Everywhere in AI

Core Search Problems—These are fundamentally about exploring sequences of actions/choices to reach a goal state.

- **Game Playing**: How should I move my chess piece?
- **Robotics**: How should a robot navigate a room?
- **Planning**: How should I schedule tasks?
- **Resource Allocation**: How should I assign tasks to processors?
- **Automated Theorem Proving**: How do I prove this mathematical statement?

Search-Assisted Problems—These primarily use other techniques (neural nets, statistical learning) but may incorporate search as a component.

- **Natural Language**: How should I parse this sentence? (May use a beam search.)
- **Machine Learning**: How should I find optimal parameters? (Gradient descent in ML.)

Hybrid/Search-Optional Problems—These can be solved with search or non-search methods. Modern approaches often blend techniques: search for exploration plus learning for evaluation.

- **Computer Vision**: How should I recognize objects in this image?
- **Web Search**: How should I rank these millions of pages?
- **Recommendation Systems**: What movie should I suggest next?
- **Drug Discovery**: Which molecular structure will treat this disease?

Formal Definition of Search

For:

S: State space (set of all possible states) $s_0 \in S$: Initial State $\text{Goals} \subseteq S$: Set of goal states

$\text{Actions}(s)$: Set of legal actions in a state s

$\delta(s, a) \rightarrow s' \in S$: Transition function δ that applies action a to state s resulting in a state s'.

$c(s, a, s')$: an action cost function; cost of $\delta(s, a) \rightarrow s'$

A search algorithm finds a sequence of actions:

$$\langle a_1, a_2, \dots, a_n \rangle \text{ such that } \delta(\dots \delta(\delta(s_0, a_1), a_2) \dots, a_n) \in \text{Goals}$$

With a cost of:

$$g(n) = \sum_{i=1}^{\text{depth}(n)} c(s_{i-1}, a_i, s_i) \qquad \text{for the path } s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} s_n$$

Formal Search Compared to DFAs

Similarities

- **Graph structure:** Both work with states and transitions
- **Path concepts:** Finding paths through state space
- **Goal-oriented:**
  - DFAs check acceptance,
  - Search finds solutions
- **State-action pairs:**
  - DFA transitions $\delta(q, a) \rightarrow q'$,
  - Search transitions $(s, a) \rightarrow s'$

Key Differences

- **Input vs. Choice:**
  - DFA reads string,
  - Search chooses actions
- **Determinism:**
  - DFA usually deterministic,
  - Search often has multiple options
- **Costs:**
  - DFA has no costs,
  - Search often optimizes path cost
- **Purpose:**
  - DFA recognizes ("Is string in language?"),
  - Search constructs ("Find path to goal")

Search problems are like “interactive DFAs” where instead of passively consuming input, we actively choose actions to reach our goal states.

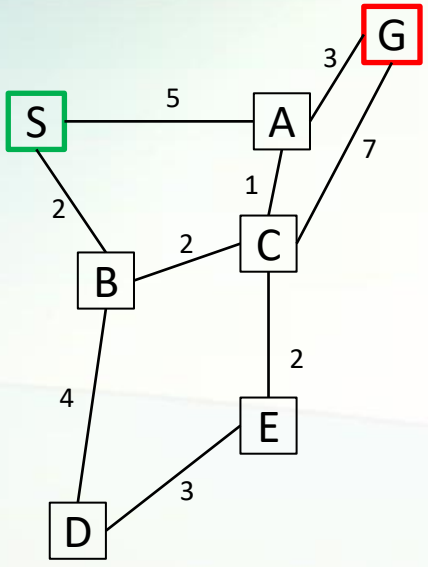
Search Example: GPS Navigation

State Space (S):	All possible locations
Initial State (δ):	Some location (e.g., home)
Goal Set (Goals):	A Mexican Restaurant
Actions(s):	TurnLeft, TurnRight, GoStraight
Transition function: $\delta(s, a) \rightarrow s'$	Given a location, s, follow a direction to end up at s'
Path Cost:	Travel time, distance, gas cost, environmental impact, etc.
Solution:	Sequence of directions

State Space Graph

For a problem, a ***state space graph*** represents all possible states as nodes, with edges as legal actions between states.

Each edge has cost $c(s, a, s')$, and multiple paths may connect states. This graph captures the complete space search algorithms explore.



An ***optimal solution*** is the path from initial state to goal with minimum total cost—not necessarily the fewest actions. Some search algorithms guarantee finding optimal solutions; others do not.

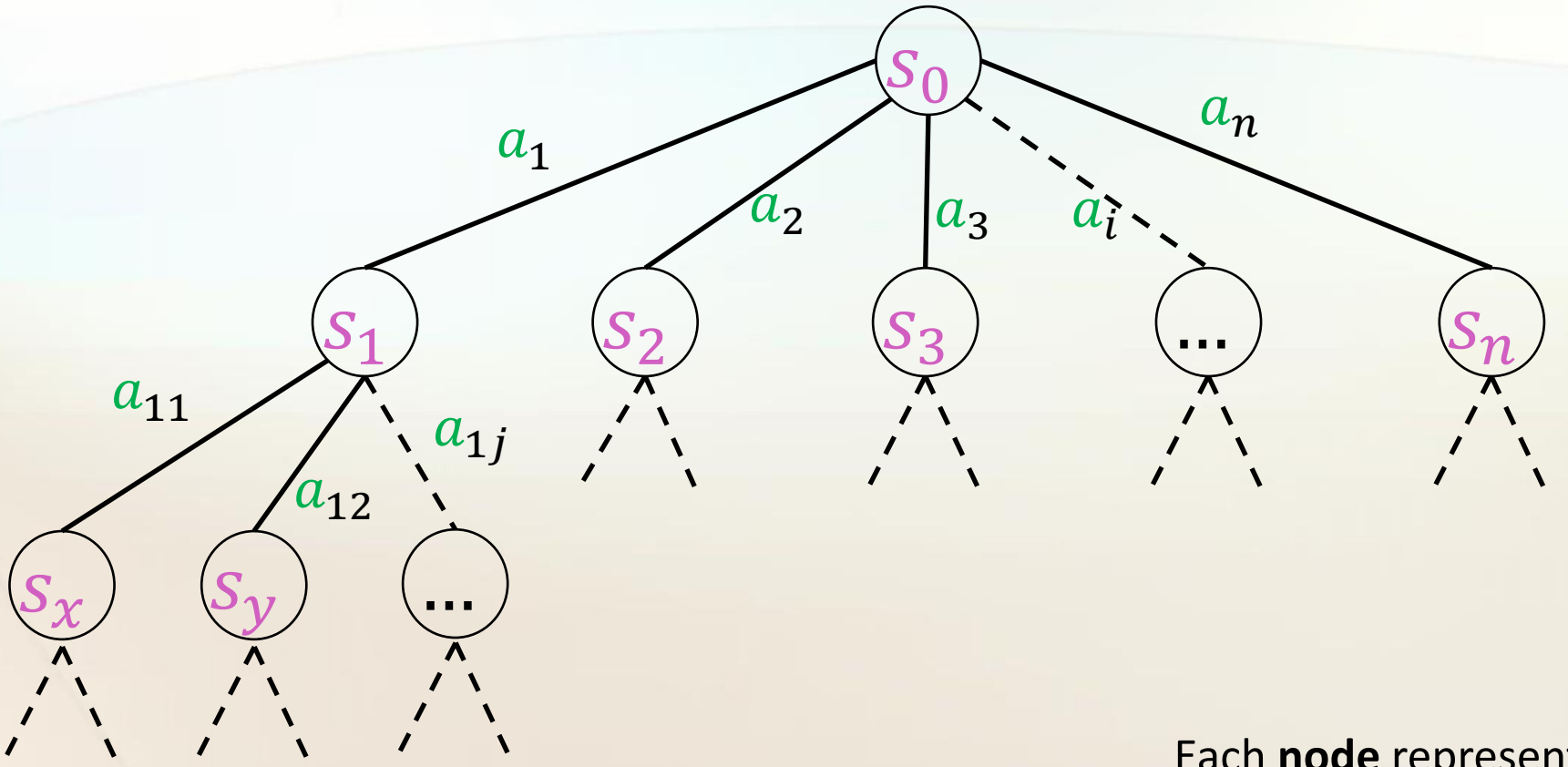
The goal of search algorithms is to navigate the state space graph efficiently to find optimal (or acceptable) solution paths.

Paths to Goal:	**Cost:**
S → A → G	5 + 3 = **8**
S → B → C → G	2 + 2 + 7 = **11**
S → A → C → G	5 + 1 + 7 = **13**
S → B → D → E → C → G	2 + 4 + 3 + 2 + 7 = **18**

Optimal means minimum cost, not fewest actions.

Search Trees

Given that the state space graph shows multiple paths to each state with different costs, search algorithms use a tree representation where each node tracks the complete path taken to reach that state.



Each **node** represents a state
Each **edge** represents an action
Each **path** represents a sequence of actions

Types of Search Problems I: Environment Properties

Search by Environment Properties refers to what we know about the problem:

- **Observability:** Can we see the current state completely?
- **Determinism:** Do actions have predictable outcomes?

Types:

Single-State Problems (Fully observable, deterministic):

- 8-puzzle: See all tiles, moves are predictable
- Route planning: Know current location, roads are fixed

Multiple-State Problems (Partially observable, deterministic):

- Robot localization: Limited sensors, do not know exact position
- Sensorless planning: Robot vacuum with no position sensors

Contingency Problems (Observable, nondeterministic):

- Medical diagnosis: Can observe symptoms, treatment outcomes uncertain
- Game against opponent: See game state, opponent's moves unpredictable

Exploration Problems (Unknown state space):

- Mars rover navigation: Don't know terrain beforehand
- Web crawling: Discover links as you go

Types of Search Problems II: Objective

Search By Objective refers to what *type of solution* we want:

- Any valid solution vs. the best solution
- One answer vs. exploring all possibilities
- Final answer vs. improving answers over time

Types:

Finding Any Solution:

- Maze solving: Any path to exit
- Satisfiability (SAT): Any assignment that satisfies clauses
- Scheduling: Any valid assignment of tasks to resources

Finding Optimal Solution:

- Shortest path: GPS navigation finding fastest route
- Traveling salesman: Minimum-cost tour visiting all cities
- Resource allocation: Minimum cost assignment

State-Space Search vs. Path-Cost Search:

- Some problems care about reaching goal states (puzzles)
- Others care about path taken (navigation, planning sequences)

Single Solution vs. All Solutions:

- Chess: Find one good move
- Constraint satisfaction: Sometimes need all valid solutions
- Game tree analysis: Explore all possible outcomes

Search Strategies

Some search algorithms are like exploring a new city without a map (systematic but blind), while others are like having a compass pointing toward your destination (guided exploration).

Uninformed Search (Blind Search)

- No problem-specific knowledge beyond definition
- Only knows actions, goal test, and action costs

BFS: Explore level by level
DFS: Go deep first, backtrack when stuck
Uniform Cost Search: Always expand lowest-cost path

Informed Search (Heuristic Search)

- Uses problem-specific knowledge (heuristic function)
- Estimates "goodness" of states

A*: Combines actual cost + estimated remaining cost
Greedy Best-First: Always move toward seemingly closest goal
Hill Climbing: Always take the best local step

Tree Search vs. Graph Search

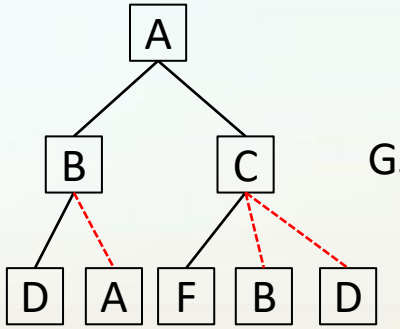
Tree Search:

- Generates nodes without checking for duplicates
- Can visit same state multiple times
- Simpler implementation

Graph Search:

- Maintains explored set to avoid repeated states
- More efficient for state spaces with many paths to same state
- Higher memory requirements

TS: Duplicate visits possible.



GS: visit each state once

(Like a forgetful housekeeper who cleans the same room multiple times.)

(Like checking off rooms on a list as you clean them.)

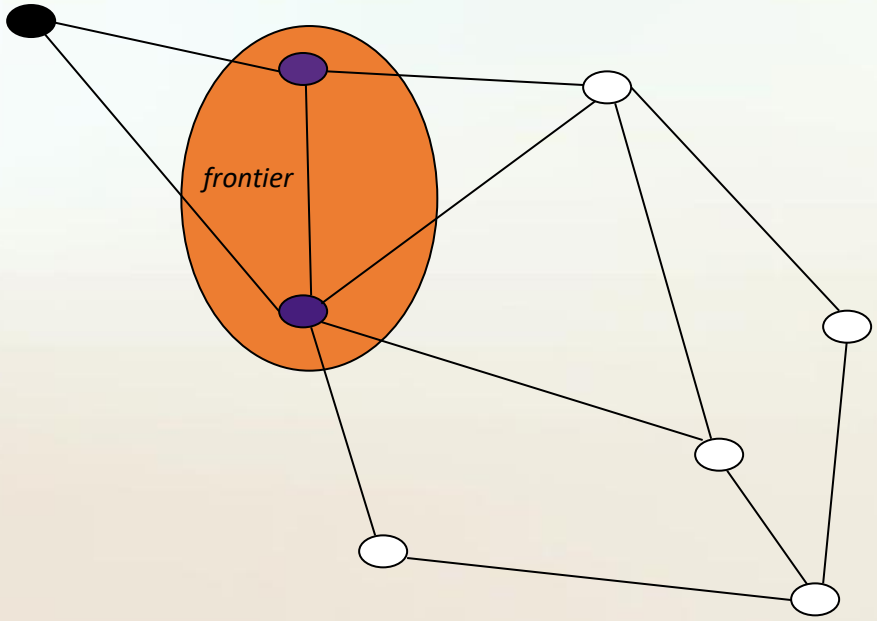
Which approach would you use?

8-puzzle (many paths to same state)?	Graph Search - avoids revisiting solved positions
Infinite state space?	Depends - Graph search if cycles exist, Tree search if linear paths
Robot navigation in known environment?	Graph Search - prevents revisiting same locations
Exploring unknown territory?	Tree Search - unknown if cycles exist, save memory

The Frontier in Search

Definition. The ***frontier*** (sometimes called fringe) is the set of all nodes that have been **identified** but not yet **expanded** in a search algorithm: leaf nodes in a search tree.

(It represents a "boundary" between explored and unexplored parts of the search tree.)



The General Search Algorithm

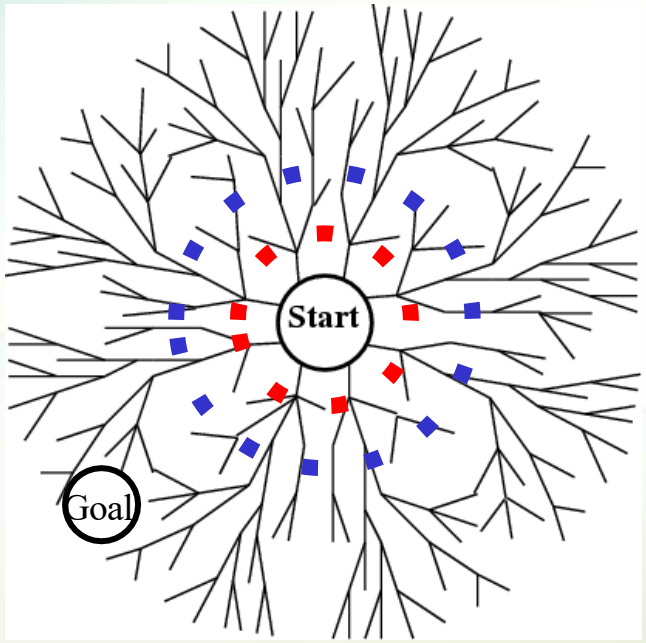
```
# Generic search algorithm
frontier = [initial_state]
explored = set()
while frontier:
    node = frontier.pop()      # Central component to Search
    if goal_test(node.state): return node
    explored.add(node.state)
    for action in actions(node.state):
        child = child_node(node, action)
        if child.state not in explored and child not in frontier:
            frontier.add(child)
```

Data structure determines the search strategy

BFS

BFS: Expand the *shallowest* node first

- Examine states **one** step away from the initial states
- Examine states **two** steps away from the initial states
- … rippling outward




```
# Generic search algorithm
frontier = [initial_state]
explored = set()
while frontier:
    node = frontier.pop() # Central component to Search
    if goal_test(node.state): return node
    explored.add(node.state)
    for action in actions(node.state):
        child = child_node(node, action)
        if child.state not in explored and child not in frontier:
            frontier.add(child)
```

BFS: Frontier = FIFO queue

We need back pointers to recover the solution path.

Performance

Four measures of search algorithms:

Completeness: Does the algorithm guarantee finding a solution if one exists? (Not necessarily finding all solutions.)

Optimality: Does it find the best solution?

Time complexity: How runtime scales with problem size.

Space complexity: How much memory is required.

BFS Performance

Completeness: Does the algorithm guarantee finding a solution if one exists? **Yes.**

Optimality: Does it find the best solution?

Yes, if edges are positive and monotonically increasing in depth:

$$cost(path\ to\ depth\ d) \leq cost(path\ to\ depth\ d+1).$$

Otherwise, no.

Solution: Uniform Cost Search

Time complexity

Worst case: goal is the last node at radius (depth) d.

- Explore all nodes up to and at radius d:

$$1 + b + b^2 + b^3 + \cdots + b^d = \frac{1(1 - b^{d+1})}{1 - b} \in O(b^d)$$

where b is the branching factor.

Space complexity

- Back pointers for generated nodes: $O(b^d)$
- Queue / fringe: smaller, but still $O(b^d)$

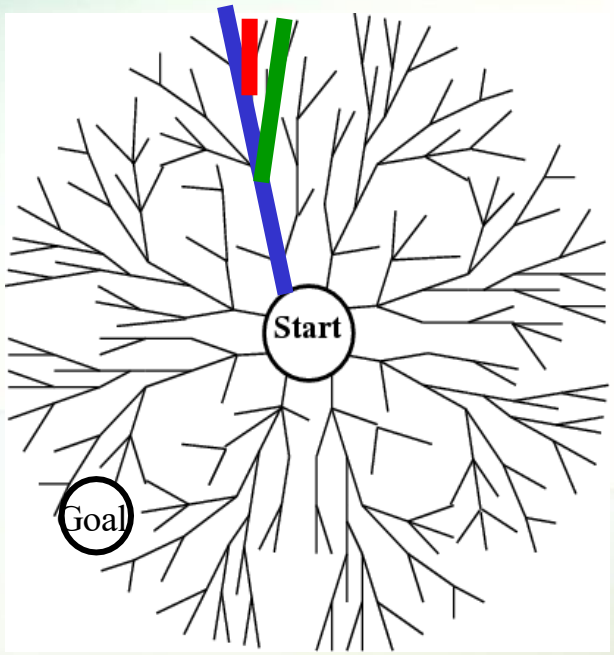
(bad)

Solution: Depth-First Search

DFS

BFS: Expand the *deepest* node first

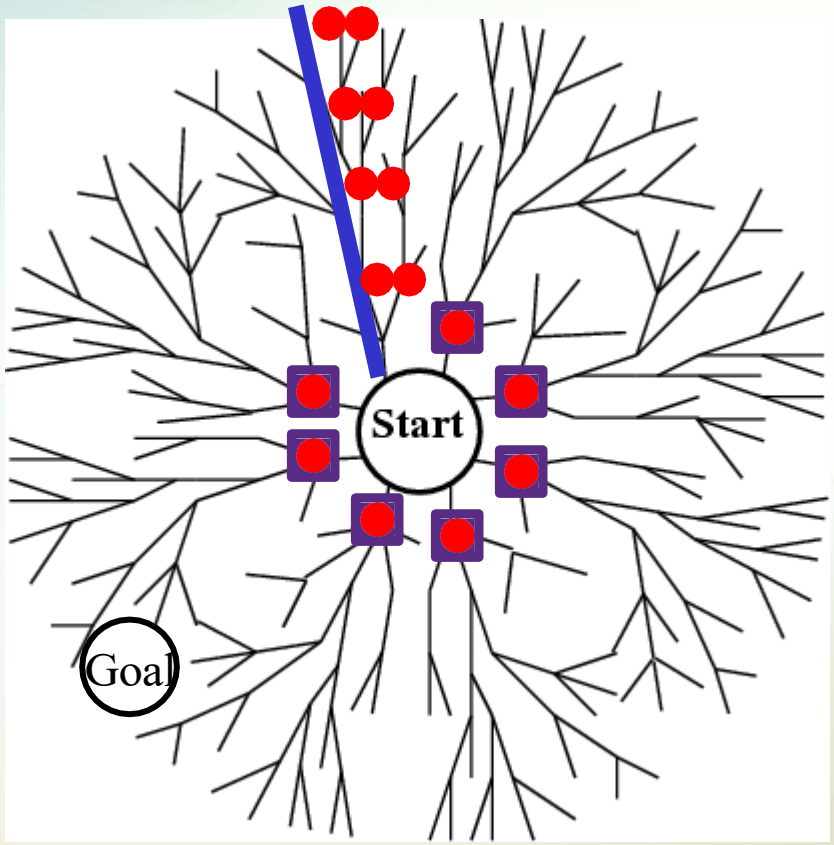
- Select a direction. Go deep until the end.
- Slightly change the end.
- Slightly change the end some more… fanning outward




```
# Generic search algorithm
frontier = [initial_state]
explored = set()
while frontier:
    node = frontier.pop() # Central component to Search
    if goal_test(node.state): return node
    explored.add(node.state)
    for action in actions(node.state):
        child = child_node(node, action)
        if child.state not in explored and child not in frontier:
            frontier.add(child)
```

DFS: Frontier = LIFO stack

DFS Fringe

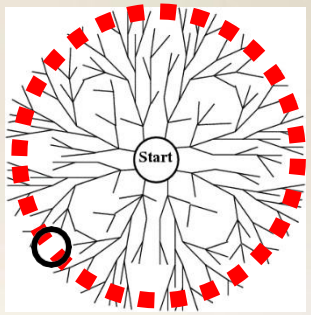


Let m be the maximum depth of the graph from **Start**.

Space complexity $m \cdot (b-1) \in O(mb)$

- The current path (up to depth m)
- The siblings of nodes along that path (up to b per level)
- With backtracking using the system stack, we do not explicitly use back-pointers.

Time complexity For a finite tree, we may visit all nodes: $O(b^m)$



BFS space + time: $O(b^d)$

The Problem with DFS

Finite tree:

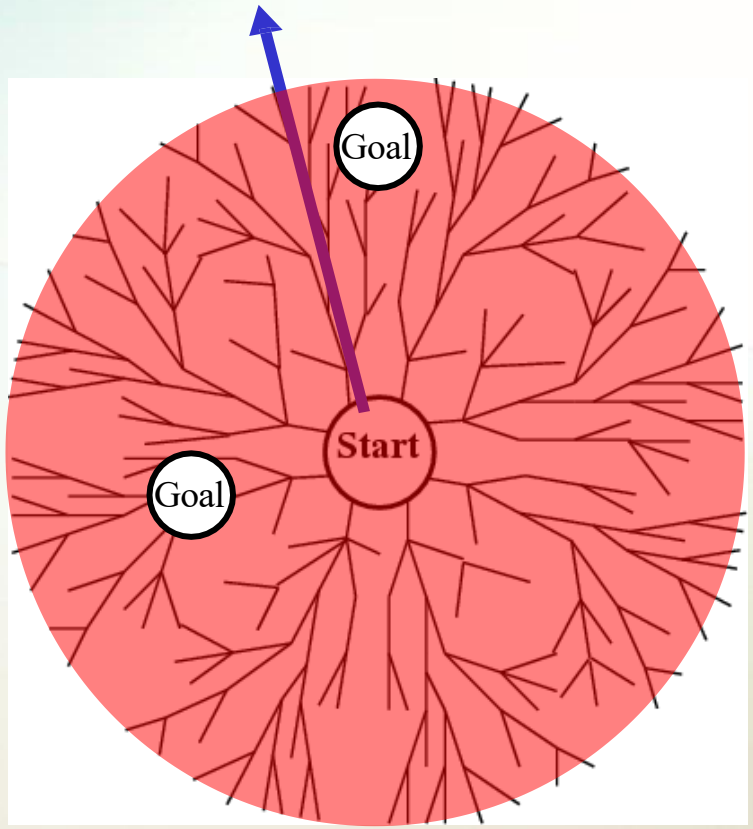
Complete — **Yes.**

Optimal — **No. Stops when the first goal is reached which is not guaranteed to be the best.**

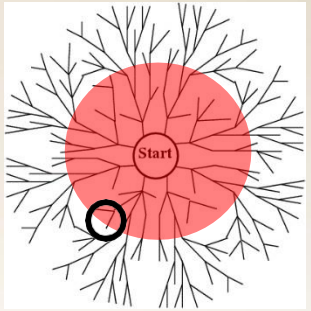
Infinite tree:

Complete — **No. May explore infinitely without backtracking toward solution.**

Optimal — **No.**



BFS



Questions

You need to search a randomly generated state space graph with one goal, uniform edge costs, depth of goal $d = 2$, and maximum graph depth $m = 100$. Considering worst-case performance, would you choose BFS or DFS for your search?

BFS

DFS could waste enormous time exploring deep, irrelevant paths: the goal is very shallow ($d = 2$) compared to maximum depth ($m = 100$).

You need to search a randomly generated state space graph with one goal, uniform edge costs, goal depth $d = 95$, and maximum graph depth $m = 100$. The branching factor is very large ($b = 1000$). Considering worst-case behavior, would you choose BFS or DFS for your search?

DFS

BFS is computationally infeasible:

Space complexity: $O(1000^{95})$
Time complexity: $O(1000^{95})$

Uniform-Cost Search (UCS)

Suppose we have a graph in which each node has a path cost (i.e., sum of edge costs along the path) from the start node. We want to find a least-cost path to a goal.

- Expand the least-cost node first
- Use a priority queue to take the least-cost item ← This sounds familiar.

UCS and Dijkstra ***Similarities***:

- Explore nodes in order of lowest cumulative cost.
- Use priority queues for minimum-cost selection
- Guarantee optimal paths

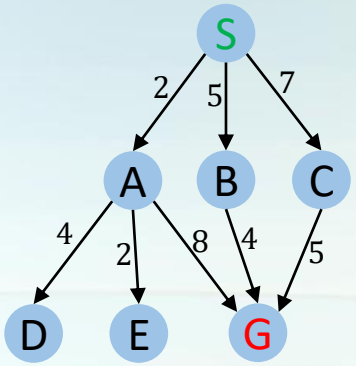
Differences

	Dijkstra's Algorithm	**Uniform Cost Search**
Implementation	Pre-load all vertices in priority queue	**Lazy insertion** - add nodes dynamically
Memory Usage	Stores entire graph upfront	Memory-efficient, on-demand generation

UCS is ideal for AI search problems with large or dynamically generated state spaces that cannot be fully stored in memory.

Example

Start: S
Goal: G



Consider successor nodes from left to right when there is a tie.

Removed		PQ	
S		S, 0	
A	A, 2	B, 5	C, 7
E	D, 6	E, 4	G, 10
B			G, 9
D			
C			
G			

Nodes visited: S A E B D C G

Path: S B G

Uniform-Cost Search

Complete? **Yes.**

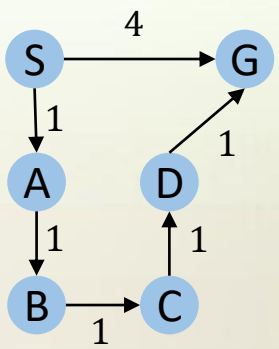
Optimal? **Yes.**

Let:

- C^* be the cost of the least-cost goal node.
- ϵ be the minimum edge cost in the graph: $edge\text{-}costs \geq \epsilon > 0$

In the worst-case, UCS may need to explore paths that use many small-cost edges. That is, to accumulate cost C^* using only a minimum-cost number of edges ϵ, we need $\frac{C^*}{\epsilon}$ edges:

$$C^* = \epsilon \cdot num(edges) \quad \Leftrightarrow \quad num(edges) = \frac{C^*}{\epsilon}$$



UCS may explore all nodes up to depth ${}^{C^*}/_{\epsilon}$:

$$1 + b + b^2 + b^3 + \cdots + b^{C^*/\epsilon} \in O\left(b^{C^*/\epsilon}\right)$$

Time complexity $O\left(b^{C^*/\epsilon}\right)$ **Space complexity** $O\left(b^{C^*/\epsilon}\right)$

UCS expands nodes by cost, not depth, and may follow deep low-cost paths before reaching a shorter but costlier goal path.

Performance Measures for Search

Completeness	Does it always find a solution if one exists?
Optimality	Does it find a least-cost solution?
Time Complexity	Number of nodes generated/expanded
Space Complexity	Maximum memory needed

Complexity

- b: Branching factor (maximum actions per state)
- d: Depth of shallowest goal node
- m: Maximum path length in state space

Most algorithms have exponential complexity.

Since no search algorithm excels in all measures of time, space, and optimality, choose based on problem-specific constraints and priorities.

Algorithm	Time	Space	Optimal?	Complete?
BFS	$O(b^d)$	$O(b^d)$	Yes*	Yes
DFS	$O(b^m)$	$O(bm)$	No	No
UCS	$O(b^{C/\epsilon})$	$O(b^{C/\epsilon})$	Yes	Yes, assuming $\epsilon > 0$

* Positive, monotonically increasing edge weights.

Questions to Consider

Why can we not always do brute-force exhaustive search?

Exhaustive search fails due to exponential growth $O(b^d)$, memory constraints, prohibitive time requirements, and the fact that finding any good solution often suffices.

What makes some search problems exponentially harder?

Problems become exponentially harder with high branching factors, deep solutions, massive state spaces (like chess), and absence of good heuristics to guide search.

How do we balance solution quality vs. computational resources?

Balancing quality versus resources involves choosing appropriate algorithms, setting time/memory limits, applying heuristics, and accepting suboptimal solutions when necessary.

When might DFS be preferred over BFS?

DFS is preferred when memory is limited ($O(bm)$ vs. $O(b^d)$), solutions are likely deep, any solution suffices, or cycles are unlikely in tree-structured problems.

What information would help guide our search efficiently?

Efficient search needs heuristics estimating goal distance, domain knowledge, cost estimates, and constraint information - motivating **informed search algorithms** like A*.

Depth-Limited Search (DLS)

What is the problem with DFS?

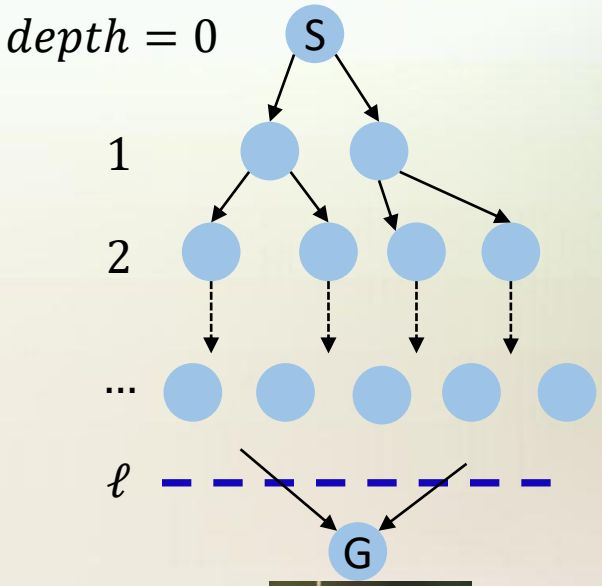
Infinite state spaces

Solution? Define a depth **limit** (ℓ) in which nodes beyond depth ℓ are not expanded.

Observe, DLS $\equiv$ DFS when $\ell = \infty$.

Time complexity $O(b^{\ell})$

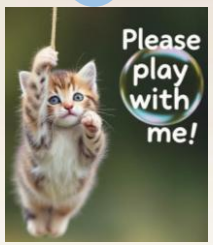
Space complexity $O(b\ell)$



What is the problem with depth-limited DFS?

Incomplete if $depth(goal) > \ell$.

What if we iterate by increasing the value of ℓ?



Iterative Deepening

What if we had the best of DFS and BFS?

Memory Efficiency of DFS: stores only current path in memory thus linear space complexity $O(bd)$.

Completeness guarantee of BFS: systematically explores all depths and will not get stuck in infinite paths.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

Figure 3.18 The iterative deepening search algorithm, which repeatedly applies depth-limited search with increasing limits. It terminates when a solution is found or if the depth-limited search returns *failure*, meaning that no solution exists.

Problem? What about all the redundancy in which we revisit nodes?

IDS: Mathematically Reasonable Redundancy

Iterative deepening performs depth-limited searches with limits $0, 1, 2, \ldots, d$.

A node at depth j will be visited each iteration from j to d: exactly $d - j + 1$ times.

Suppose $d = 5$.
A node at depth $j = 2$ appears in searches with limits 2, 3, 4, 5 → 4 times.

Therefore, the total number of node visits for IDS is given by:

$$N(IDS) = b^0 \cdot (d+1) + b^1 \cdot d + b^2 \cdot (d-1) + \cdots + b^d \qquad \in O(b^d)$$

Intuition. With exponential expansion, the deepest level dominates the computation:

$$N(nodes\ up\ to\ depth\ d) = b^0 + b^1 + \cdots + b^{d-2} + b^{d-1} = \frac{b^d - 1}{b-1} \in O(b^d)$$

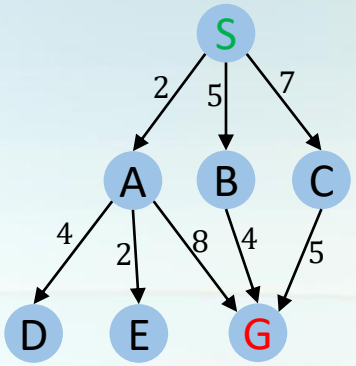
Asymptotically, the redundant work at shallow levels becomes insignificant compared to the work done at the target depth.

So IDS is $O(b^d)$; same as BFS.

Iterative deepening is the preferred uninformed search method when the search state space is larger than can fit in memory and the depth of the solution is not known.

Example: Nodes Expanded by Search

Start: S
Goal: G



Consider successor nodes from left to right when there is a tie.

What nodes are visited and what solutions are found by the search technique?

Search	Nodes Visited	Solution Found
BFS	S A B C D E G	S A G
DFS	S A D E G	S A G
IDS	S S A B C S A D E G	S A G
UCS	S A E B D C G	S B G

(UCS is the only uninformed search that worries about costs.)